

Ella Potter & Adrian Madrigal

Professor Beichel

ECE:3360 Embedded Systems

Post-Lab Report 2

1. Introduction

The main goal for this lab was to create a hexadecimal up/down counter using a 7-segment display that receives input from an 8-bit shift register connected to the microcontroller. The counter should increase/decrease by one when a user presses a push button for less than one second, it should switch between increment/decrement mode if the button is pressed for more than one but less than two seconds, and the counter should reset if the button is held for over 2 seconds.

2. Schematic

Figure 1 shows the circuit set-up for the 7-segment display, shift register, and microcontroller. PB0, PB1, and PB2 are all used as outputs to send info to the shift register to display the digits on the 7-segment display. To implement the push button, the circuit given in the ES25_Lab02 document (included in the resources section) was used and then connected to PB3 on the microcontroller as an input.

*Note: The resistors used in the circuit are 560 ohms

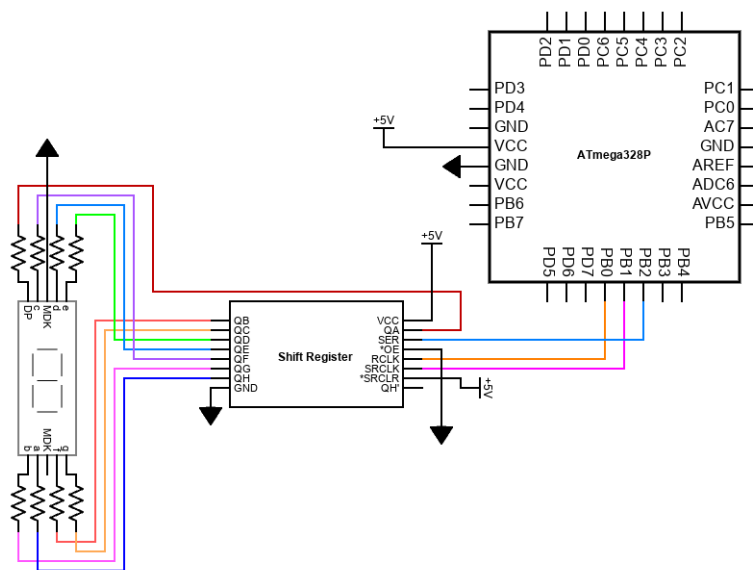


Figure 1: 7-Segment Display Circuit

3. Discussion

This lab consisted of two main parts to be able to complete the requirements: building the circuit and writing the code to control the circuit.

3.1 Hardware

The first thing that needed to be done was to build a circuit using the ATmega328P microcontroller, the SNx4HC595 8-bit Shift Register, the 5161AS 7-segment display, and the push-button.

- **ATmega328P**: used both to process input from the push-button to know when the user pressed the button and to send output to the shift register which is then used to control the 7-segment display and display a digit
- **SNx4HC595**: used to send 8-bit data to the 7-segment display
 - **SER**: carries the 8-bit data bit by bit from the microcontroller into the shift register
 - **SRCLK**: moves data into the register on each rising clock edge
 - **RCLK**: outputs the data to the 7-segment display which all bits are loaded in
- **5161AS**: used to display the hexadecimal number, each display segment is controlled by a corresponding pin which will receive either a 0 or 1 from the shift register
- **Push-Button**: used by the user to control the 7-segment display
 - **Button Press < 1 sec**: increase/decrease counter by one
 - **1 sec < Button Press < 2 sec**: switch between increment/decrement mode
 - **Button Press > 2 sec**: reset the counter to 0 or F
- **Resistors**: used by both the 7-segment display and the push button to limit the current flow to the display and to debounce the push button circuit
 - To calculate value of resistors for 7 segment display: $5\text{ V} - 1.8\text{ V} / 6\text{ mA}$

3.2 Software

After setting up the circuit, the logic for the microcontroller was written using AVR assembly language and needed to be able to implement the following functions:

Function	Description
Main Loop	The code should continuously loop until the user presses the button
Button Counter	When the user presses the push button the code should use a delay function to keep track of how long the button is pressed down for

Display	If the button is pressed for less than one second, the code should send the correct data to the shift register to display a hexadecimal digit
Mode Selection	If the button is pressed for between one and two seconds the code should change the current mode the counter is in. i.e. if counter is currently incrementing it should begin decrementing
Reset	If the button is pressed for over two seconds the code should reset the display value to 0 if in increment mode and to F if in decrement mode and then display it
Increment/Decrement	Need to keep track of what mode the counter is in and send the correct hexadecimal number for that mode. (When decrementing the DP light should be lit up)

Another part of the software design included calculating the correct values to send to the shift register to display the digits 0-F. The datasheet for the 5161AS contains a diagram that shows what pins light up what segments on the display. Using that diagram and looking at our circuit to find the associated pins on the shift register you can calculate the 8-bit binary value needed to light up each hexadecimal digit and then convert that binary value to hex. To keep track of the values needed to display each digit, the following table was created:

	Increment Mode		Decrement Mode	
Digit	Binary Representation	Hex Value	Binary Representation	Hex Value
0	1111 1010	FA	1111 1011	FB
1	0110 0000	60	0110 0001	61
2	1101 1100	DC	1101 1101	DD
3	1111 0100	F4	1111 0101	F5
4	0110 0110	66	0110 0111	67
5	1011 0110	B6	1011 0111	B7
6	1011 1110	BE	1011 1111	BF
7	1110 0000	E0	1110 0001	E1
8	1111 1110	FE	1111 1111	FF
9	1110 0110	E6	1110 0111	E7
A	1110 1110	EE	1110 1111	EF
B	0011 1110	3E	0011 1111	3F
C	1001 1010	9A	1001 1011	9B
D	0111 1100	7C	0111 1101	7D
E	1001 1110	9E	1001 1111	9F
F	1000 1110	8E	1000 1111	8F

These values were then stored in two separate look-up tables, inc_table and dec_table, which were used to load R16 with the correct data to send to the shift register.

4. Conclusion

From this lab, I have gained more experience in how to code the microcontroller to send outputs to a shift register as well as take in and keep track of input from a push-button. The circuit design was the easiest and fastest part of this lab while the software design and the logic behind it is what took a majority of the time. Being able to correctly receive input from the button and keep track of how long it was held as well as sending the correct output at the correct time took a lot of debugging. This led to a lot more experience gained in how to debug using Microchip Studio and assembly language. Overall, this lab helped me fully understand the AVR assembly language.

5. Appendix A: Source Code

```
.include "m328Pdef.inc"

.cseg

.org 0

sbi DDRB,0    ; PB0 = SER output (to SN74HC595)

sbi DDRB,1    ; PB1 = SRCLK output (to SN74HC595)

sbi DDRB,2    ; PB2 = RCLK output (to SN74HC595)

cbi DDRB,3    ; PB3 = Button input (hardware debounced)

ldi R18, 0x1    ; Used to increment Z register to find lookup table values

ldi R19, 0x00    ; Used to keep track of number of button presses

ldi R27, 0x00    ; Used to keep track of how long button is pressed

ldi R20, 0x0    ; Used to keep track of mode (0: increment, 1:decrement)

.equ count = 0x8FF

ldi ZH, high(inc_table << 1) ; Load high byte of lookup table (initially in increment mode)

ldi ZL, low(inc_table << 1) ; Load low byte of lookup table

lpm R16, Z    ; Load 7-segment pattern for 0

rcall display ; Show "0" initially


main_loop:

    sbic PINB, 3    ; Loop until button is pressed

    rjmp main_loop

    ldi R27, 0x00    ; Set R27 to 0 to start counting
```

```
rjmp wait_release ; Go to wait release to count
```

```
reset: ;resets all registers to initial values to restart display values
```

```
ldi R19, 0x00
```

```
cpi R20, 0x1
```

```
breq dec_reset ; resets decrement table
```

```
brne inc_reset ;resets increment table
```

```
dec_reset:
```

```
ldi ZH, high(dec_table << 1)
```

```
ldi ZL, low(dec_table << 1)
```

```
lpm R16, Z ; Load 7-segment pattern for 0
```

```
rcall display ; show original value again
```

```
rjmp main_loop
```

```
inc_reset:
```

```
ldi ZH, high(inc_table << 1)
```

```
ldi ZL, low(inc_table << 1)
```

```
lpm R16, Z ; Load 7-segment pattern for 0
```

```
rcall display ; show original value again
```

```
rjmp main_loop
```

```
update_display:
```

```
cpi R19, 0x10 ; If button press > 16
```

```
brsh reset ; Reset display to 0
```

```
add ZL, R18 ; increment ZL by 1 to load next table value
```

```
lpm R16, Z ; Load segment pattern from lookup table
```

```
rcall display ; Call display function to update 7-segment
```

```
rjmp main_loop ; jump back to loop to wait for button press
```

```
mode_select:
```

```
cpi R20, 0x0
```

```
breq dec_mode
```

```
brne inc_mode
```

```
inc_mode:
```

```
ldi R20, 0x0
```

```
ldi R19, 0x00
```

```
ldi ZH, high(inc_table << 1)
```

```
ldi ZL, low(inc_table << 1)
```

```
lpm R16, Z ; Load 7-segment pattern for 0
```

```
rcall display ; Show "0" initially
```

```
rjmp main_loop
```

```
dec_mode:
```

```
ldi R20, 0x1
```

```
ldi R19, 0x00
```

```
ldi ZH, high(dec_table << 1)
```

```
ldi ZL, low(dec_table << 1)
```

```
lpm R16, Z ; Load 7-segment pattern for 0
```

```
rcall display ; Show "0" initially
```

```
rjmp main_loop
```

```
wait_release:
```

```
rcall delay
```

```
inc R27 ; count how long button held
```

```
sbis PINB, 3 ; Wait for button release
```

```
rjmp wait_release ; Keep looping until button release
```

```
inc R19 ; Increase button counter by one
```

```
cpi R27, 0x3D ; see if button was pressed for over 1 second
```

```
brlo update_display ; if less then 1 second update display
```

```
cpi R27, 0x7A ; see if button pressed for over 2 seconds
```

```
brlo mode_select ; greater then 1 sec but less then 2 goes to mode select
```

```

        brsh reset

delay:      ;delay function from Lab1

        ldi r28, 0xFF          ; changed registers to avoid conflict

d1:         ; with Z register holding seg_table values

        ldi r29, 0xFF

d2:

        nop

        dec r29

        brne d2

        dec r28

        brne d1

        ret

display:

        ; backup used registers on stack

        push R16

        push R17

        in R17, SREG

        push R17

        ldi R17, 8 ; loop --> test all 8 bits

loop:

        rol R16 ; rotate left trough Carry

        BRCS set_ser_in_1 ; branch if Carry is set

        cbi PORTB, 0; put code here to set SER to 0

        ...

        rjmp end

set_ser_in_1:

        sbi PORTB, 0; put code here to set SER to 1

        ...

end:

```

```

sbi PORTB, 1

cbi PORTB, 1; put code here to generate SRCLK pulse

...

dec R17

brne loop

sbi PORTB, 2

cbi PORTB, 2; put code here to generate RCLK pulse

...

; restore registers from stack

pop R17

out SREG, R17

pop R17

pop R16

ret

inc_table:

.db 0xFA, 0x60, 0xDC, 0xF4, 0x66, 0xB6, 0xBE, 0xE0

.db 0xFE, 0xE6, 0xEE, 0x3E, 0x9A, 0x7C, 0x9E, 0x8E ; Hex values to show 0-F on display

dec_table:

.db 0x8F, 0x9F, 0x7D, 0x9B, 0x3F, 0xEF, 0xE7, 0xFF

.db 0xE1, 0xBF, 0xB7, 0x67, 0xF5, 0xDD, 0x61, 0xFB

```

6. Appendix B: References

Texas Instruments, “SNx4HC595 8-Bit Shift Registers With 3-State Output Registers,” SN4HC595 datasheet, Dec. 1982 [Revised Sept. 2015].

XLITX Technology, 5161AS datasheet, www.xltix.com.

